

UNIVERSIDAD ARGENTINA DE LA EMPRESA

Ingeniería en Informática

PARADIGMA ORIENTADO A OBJETOS

Trabajo Práctico Obligatorio — Primera Entrega

ESCAPE CORPORATIVO: LA FUGA DEL EMPLEADO

Propuesta de videojuego de acción y estrategia 2D en tiempo real

Documento de Diseño de Juego y Diagrama de Clases UML

Integrantes: Alcaide Yamil; Esponda Emiliano; Miguez Ian Franco; Torres Matias; Villavedra

Santiago Martin

Profesor: Hernandez Pablo Eduardo

Fecha de entrega: 05/10/2026

1. Introducción.....	3
2. Objetivo del juego.....	4
2.1. <i>Condiciones de victoria.....</i>	<i>4</i>
2.2. <i>Condiciones de derrota.....</i>	<i>4</i>
3. Actores y entidades del sistema.....	4
3.1. <i>El Empleado.....</i>	<i>4</i>
3.2. <i>El Jefe y sus Proyectiles.....</i>	<i>5</i>
3.3. <i>El Entorno (El Laberinto).....</i>	<i>6</i>
4. Reglas de jugabilidad y mecánicas.....	6
<i>Regla 1 — Movimiento y Colisiones (Motor de Física 2D).....</i>	<i>7</i>
<i>Regla 2 — Sistema de Buffos (Polimorfismo).....</i>	<i>7</i>
<i>Regla 3 — Progresión y Persistencia.....</i>	<i>7</i>
5. Manejo de Excepciones y Eventos.....	7
6. Arquitectura técnica y decisiones de diseño orientado a objetos.....	8
6.1. <i>Lenguaje, plataforma y herramientas.....</i>	<i>8</i>
6.2. <i>Justificación de las decisiones de diseño.....</i>	<i>8</i>
7. Diagrama de clases UML.....	9
7.1. <i>Descripción de las clases principales.....</i>	<i>10</i>
8. Alcance y plan de trabajo.....	11
9. Conclusión.....	11

1. Introducción

El proyecto **ESCAPE CORPORATIVO: LA FUGA DEL EMPLEADO** propone la creación de un videojuego 2D de acción y estrategia en tiempo real desarrollado en Java. La premisa narrativa posiciona al jugador en el rol de un empleado exhausto que intenta escapar de un entorno de oficina opresivo antes de que venza su horario laboral, mientras evade incansablemente a su Jefe, quien intenta asignarle horas extra mediante proyectiles cargados de tareas.

Desde el punto de vista académico, este desarrollo sirve como caso de estudio práctico para la implementación integral de los pilares del Paradigma Orientado a Objetos (POO). El diseño del software incorpora abstracción mediante jerarquías claras de entidades, herencia y polimorfismo para la gestión de elementos e ítems, interfaces para el desacoplamiento de comportamientos, así como los principios SOLID y patrones GRASP. Adicionalmente, el sistema explota colecciones genéricas para el manejo dinámico de objetos en pantalla, un esquema robusto de excepciones personalizadas, procesamiento multihilo (Threads) para garantizar una tasa de refresco constante en el bucle principal de juego (*Game Loop*), persistencia de datos mediante serialización o JSON, y un entorno gráfico responsive construido con la biblioteca Java Swing/AWT.

2. Objetivo del juego

El objetivo central del jugador consiste en guiar al Empleado desplazándose en cuatro direcciones (izquierda, derecha, arriba, abajo) desde su escritorio (punto de origen) hasta la Salida Principal de la corporación (meta), atravesando un laberinto con muros infranqueables y esquivando los proyectiles lanzados por el Jefe.

2.1. Condiciones de victoria

- El jugador alcanza las coordenadas de la **Salida Principal** antes de que finalice la jornada laboral (tiempo límite).
- El jugador ha recolectado todos los ítems clave de salida solicitados en niveles avanzados (por ejemplo, "Tarjeta de Acceso").

2.2. Condiciones de derrota

- Un proyectil impacta contra el Empleado o este entra en contacto directo con el Jefe, lo que provoca la pérdida automática del intento y el reinicio de la posición al inicio del mapa.

3. Actores y entidades del sistema

A continuación se detallan los elementos activos y pasivos dentro del dominio del problema, clasificados según su funcionalidad e interacción dentro de la simulación.

3.1. El Empleado

El jugador interactúa directamente con el Empleado, pudiendo alterar sus propiedades dinámicamente mediante el consumo de ítems (buffos) distribuidos en el mapa.

Entidad	Rol	Comportamiento
Empleado	Personaje Principal ▾	Movido por el jugador vía teclado en cuatro direcciones (izquierda, derecha, arriba, abajo). Detecta colisiones con muros infranqueables e ítems/zonas de buffos.

3.2. El Jefe y sus *Proyectiles*

El Jefe actúa como el antagonista principal impulsado por inteligencia artificial básica, persiguiendo al jugador y lanzando obstáculos.

Entidad	Rol	Comportamiento
Jefe	Antagonista / IA	Persigue dinámicamente al Empleado evaluando su posición. Cambia entre estado de patrulla y persecución.
Pala / Tarea	Proyectil	Objeto lanzado por el Jefe hacia la ubicación del Empleado. Si impacta, causa la pérdida automática y devuelve al jugador al inicio.

3.3. El Entorno (El Laberinto)

El mapa define los límites físicos del juego y el espacio euclidiano donde ocurren las colisiones.

Entidad	Rol	Comportamiento
Mapa / Grid	Estructura de Nivel	Matriz bidimensional que organiza celdas transitables e intransitables. Cargado dinámicamente.
Muro / Cubículo	Obstáculo Rígido	Estructura rígida e infranqueable que bloquea el paso del Empleado y del Jefe, destruyendo o frenando proyectiles al colisionar.
Inicio / Meta	Puntos de Control	Marca la coordenada de aparición (<i>spawn</i>) y el punto de activación de la victoria (<i>trigger</i>).
ZonaBuffo	Punto de obtencion de buffos	Zona por la que, al pasar, el jugador obtiene un buffo de tipo Cafe o de tipo Aguinaldo

4. Reglas de jugabilidad y mecánicas

La lógica del juego se articula a través de cuatro reglas de negocio centrales traducidas a estructuras del lenguaje Java:

Regla 1 — Movimiento y Colisiones (Motor de Física 2D)

En cada ciclo del bucle de juego, la posición intentada en cuatro direcciones (izquierda, derecha, arriba, abajo) es evaluada mediante la intersección de sus cajas envolventes (Bounding Boxes / `java.awt.Rectangle`). Si existe solapamiento con un Muro infranqueable, el movimiento se anula. Si impacta un proyectil, el jugador pierde de inmediato y vuelve al punto de inicio.

Regla 2 — Sistema de Buffos (Polimorfismo)

Todos los buffos heredan de una clase abstracta `Buffo` con un contador de tiempo para acabarse. Al entrar en contacto con las zonas, el motor modifica temporalmente la velocidad de desplazamiento (con el buffo “Cafe”) o la resistencia frente a proyectiles (con el buffo “Aguinaldo”) de manera polimórfica, respetando el principio de Abierto/Cerrado (OCP).

Regla 3 — Progresión y Persistencia

El progreso del jugador, junto con la tabla de mejores tiempos (High Scores), es serializado en disco. Si finaliza un nivel exitosamente, el estado actual se consolida mediante operaciones de entrada/salida (I/O).

5. Manejo de Excepciones y Eventos

El juego contempla una estructura jerárquica de excepciones personalizadas derivando de `Exception` o `RuntimeException` para controlar situaciones anómalas sin interrumpir abruptamente el bucle de ejecución:

- **FueraDeLimitesException**: Lanzada cuando una entidad intenta calcular coordenadas fuera de los rangos válidos de la matriz del mapa.
- **JuegoTerminadoException**: Disparada cuando la salud del jugador se agota o el tiempo llega a cero, provocando la transición limpia hacia la pantalla de *Game Over*.
- **NivelNoEncontradoException**: Excepción checked lanzada por el gestor de archivos al intentar cargar una estructura de mapa inexistente o corrupta.

6. Arquitectura técnica y decisiones de diseño orientado a objetos

6.1. Lenguaje, plataforma y herramientas

- **Lenguaje**: Java SE (JDK 17 o superior).
- **Biblioteca Gráfica**: Java Swing y AWT (**JFrame**, **JPanel**, **Graphics2D**) con técnica de doble búfer (*Double Buffering*) para evitar el parpadeo (*flicker*).
- **Multihilo**: Implementación de la interfaz **Runnable** en el motor de juego para gestionar un hilo secundario exclusivo para el ciclo **tick()** y **render()**.
- **Persistencia**: **java.io.Serializable** / **Gson** para almacenamiento local de puntuaciones y estado.

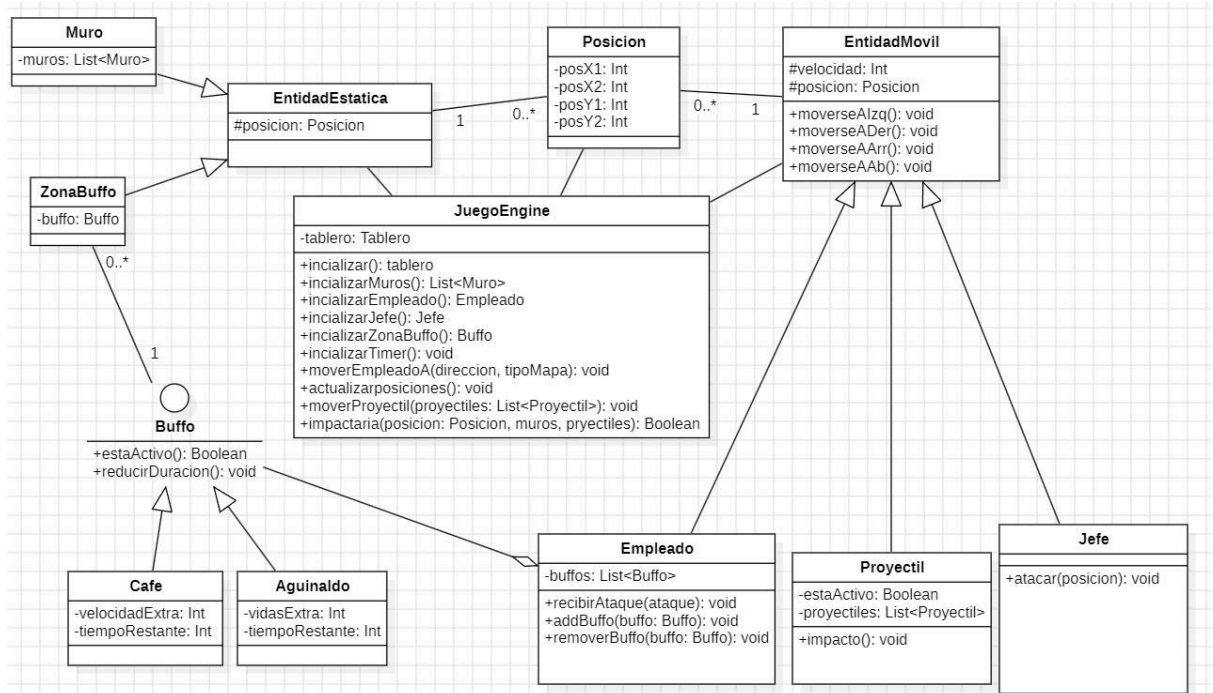
6.2. Justificación de las decisiones de diseño

Concepto de POO	Aplicación en Escape Corporativo
Abstracción	Creación de la clase abstracta EntidadJuego definiendo atributos de coordenadas (x, y), dimensiones y el método dibujar(Graphics g) .

Concepto de POO	Aplicación en Escape Corporativo
Herencia	<code>Empleado</code> , <code>Jefe</code> y <code>Proyectil</code> extienden de <code>EntidadMovil</code> , utilizando la lógica posicional y de dibujo base.
Polimorfismo	Invocación unificada del método <code>actualizar()</code> sobre una colección genérica <code>List<EntidadJuego></code> en cada fotograma.
Encapsulamiento	Atributos privados (<code>private</code>) en todas las clases clave, expuestos únicamente mediante getters/setters validados (ej. no permitir energía negativa).
Interfaces	Definición de <code>Colisionable</code> y <code>EstrategiaIA</code> para desacoplar el comportamiento de colisión y toma de decisiones de la estructura estática del objeto.

7. Diagrama de clases UML

El modelo UML se estructura considerando un desacoplamiento claro entre el modelo de dominio, la vista y el controlador del juego.



7.1. Descripción de las clases principales

- **EntidadMovil**(Clase Abstracta): Cimiento de la jerarquía gráfica e interactiva.
- **Empleado** (Clase Concreta): Representa al avatar del jugador; administra sus estados (ej. normal, acelerado) e interactúa con los controles de teclado (**KeyListener**).
- **Jefe** (Clase Concreta): Mantiene una referencia a la interfaz **EstrategiaIA** para determinar su vector de movimiento hacia el jugador en cada iteración.
- **ZonaBuffo**(Clase Abstracta): Modela objetos estáticos que reaccionan ante la colisión del jugador.
- **JuegoEngine** (Controlador Principal): Gestiona el *Game Loop*, procesa la entrada del usuario, actualiza la física y refresca el panel de Swing.

8. Alcance y plan de trabajo

Entrega	Contenido previsto	Estado
Primera entrega (actual)	Documento de diseño (GDD) y diagrama de clases UML	Entregado
Segunda entrega	Modelo de dominio en Java: clases, herencia, enum, colecciones y excepciones	Pendiente
Entrega final	Aplicación completa con GUI Swing, hilos, persistencia y defensa oral	Pendiente

9. Conclusión

El proyecto **ESCAPE CORPORATIVO: LA FUGA DEL EMPLEADO** articula de manera exhaustiva las exigencias académicas de la materia Paradigma Orientado a Objetos con un enfoque práctico y dinámico. Mediante la correcta aplicación de abstracción, herencia, polimorfismo y principios de diseño robustos, se garantiza la entrega de un código fuente escalable, mantenible y modular, demostrando el dominio técnico de Java en entornos gráficos interactivos en tiempo real.

Nota: Se utilizó asistencia de inteligencia artificial para mejorar la fluidez y redacción general del documento.